Bioinformatics
4. Decoding and Evaluation: The Viterbi and Forward-Backward Algorithms

Lecture 4 of 10

Prof. Magnus Bordewich

Durham University

Recap: Three Problems, One Tool

Lecture 3 posed three problems for an HMM: decoding, evaluation, and posterior state probabilities

All three are solved by dynamic programming, exploiting the same Markov structure that made exhaustive enumeration exponential

Today: Viterbi's algorithm (decoding), then the forward and backward algorithms (evaluation and posteriors)

A Running Example: a Two-State HMM

States F (fair coin) and B (biased)

Transitions $m_{FF} = 0.75, m_{FB} = 0.25, m_{BF} = 0.25, m_{BB} = 0.75$

Emissions $e_F(H) = 0.5, e_F(T) = 0.5, e_B(H) = 0.75, e_B(T) = 0.25$

Equal start probabilities

Observed sequences $\boldsymbol{x} = TTH$ and $\boldsymbol{x} = TTHHH$

Viterbi's algorithm

PROBLEM 1: THE MOST PROBABLE STATE PATH

What We Want to Compute

Given observed $x = (x_1, \ldots, x_L)$, find the state path s^* that maximises the probability $p(s \mid x)$

Since $p(s|x) = p(s,x) \,/\, p(x)$, and $p(x)$ does not depend on s, this is the same as maximising the joint probability $p(s,x)$

$$s^* = argmax_{s1,\ldots,sL} \{p(s_1, \ldots, s_L;\ x_1, \ldots, x_L)\}$$

$$= argmax_{s1,\ldots,sL} \left\{ \prod_{i=1}^{L} m_{s_{(i-1)},s_i} \,.\, e_{s_i}(x_i) \right\}$$

Viterbi's Algorithm: The Key Quantity

For position $i = 1, \ldots, L$ and each state l, define:

- $v_l(i) =$ the probability of the single most probable path through positions $1..i$ that ends in state l, jointly with emitting $x_1, \ldots, x_i$

Because of the Markov property, $v_l(i)$ can be built from the $v_k(i-1)$ values for the previous position:

$$v_l(i) = e_l(x_i) \max_k [\, v_k(i-1)\, m_{kl} \,]$$

Viterbi's Algorithm: Forward calculation and back tracking

Add prior estimates of $p(s_1 = l)$, e.g. uniform or by standard frequency

Set $v_l(1) = e_l(x_1)p(s_1 = l)$ for each state l

For $i = 2, \ldots, L$:

- For each state $l, v_l(i) = e_l(x_i) \max_k [\, v_k(i-1)\, m_{kl} \,]$
- And $prev_l(i) = \operatorname{argmax}_k [\, v_k(i-1)\, m_{kl} \,]$

(remember the best predecessor)

Result: the optimal probability is $\max_k v_k(L)$;

recover s^* by tracing back through $prev$

Worked Example: Decoding the Dishonest Casino

States F (fair coin) and B (biased)

Transitions $m_{FF} = 0.75, m_{FB} = 0.25, m_{BF} = 0.25, m_{BB} = 0.75$

Emissions $e_F(H) = 0.5, e_F(T) = 0.5, e_B(H) = 0.75, e_B(T) = 0.25$

Equal start probabilities

Observed sequences $\boldsymbol{x} = TTH$ and $\boldsymbol{x} = TTHHH$

State	t1: T	t2: T	t3: H
F	$v_F(1) = p(s_1 = F).e_F(T)$	$v_F(2) = e_F(T) \cdot max\,[v_F(1)m_{FF}, v_B(1)m_{BF}\,]$	$v_F(3) = e_F(H) \cdot max\,[v_F(2)m_{FF}, v_B(2)m_{BF}\,]$
B	$v_B(1) = p(s_1 = B).e_B(T)$	$v_B(2) = e_B(T) \cdot max\,[v_F(1)m_{FB}, v_B(1)m_{BB}\,]$	$v_B(3) = e_B(H) \cdot max\,[v_F(2)m_{FB}, v_B(2)m_{BB}\,]$

Worked Example: Decoding the Dishonest Casino

States F (fair coin) and B (biased)

Transitions $m_{FF} = 0.75, m_{FB} = 0.25, m_{BF} = 0.25, m_{BB} = 0.75$

Emissions $e_F(H) = 0.5, e_F(T) = 0.5, e_B(H) = 0.75, e_B(T) = 0.25$

Equal start probabilities

Observed sequences $\boldsymbol{x} = TTH$ and $\boldsymbol{x} = TTHHH$

State	t1: T	t2: T	t3: H	t4: H	t5: H
F	2.2^{-3}	12.2^{-7} (F)	72.2^{-11} (F)	432.2^{-15} (F)	2592.2^{-19} (F)
B	1.2^{-3}	3.2^{-7} (B)	36.2^{-11} (F)	324.2^{-15} (B)	$\mathbf{2916.2^{-19}}$ (B)

Most likely hidden sequence: FFBBB

Worked Example: Decoding the Dishonest Casino

States F (fair coin) and B (biased)

Transitions $m_{FF} = 0.75, m_{FB} = 0.25, m_{BF} = 0.25, m_{BB} = 0.75$

Emissions $e_F(H) = 0.5, e_F(T) = 0.5, e_B(H) = 0.75, e_B(T) = 0.25$

Equal start probabilities

Observed sequence $\boldsymbol{x} = TTHHT$

State	t1: T	t2: T	t3: H	t4: H	t5: T
F	2.2^{-3}	12.2^{-7} (F)	72.2^{-11} (F)	432.2^{-15} (F)	$\mathbf{2592.2^{-19}}$ (F)
B	1.2^{-3}	3.2^{-7} (B)	36.2^{-11} (F)	324.2^{-15} (B)	972.2^{-19} (B)

Most likely hidden sequence: FFFFF

Practicalities: Log-Space, and Complexity

Multiplying many probabilities quickly underflows floating point

In practice Viterbi works in log-space:

$$\log v_l(i) = \log e_l(x_i) + \max_k [\log v_k(i-1) + \log m_{kl}]$$

(max is unaffected since log is monotonic)

Time: $O(m^2 L)$ - for each of L positions and m states, a max over m predecessors

Space: $O(mL)$ for traceback or $O(m)$ if only the final probability is needed

The forward algorithm

PROBLEM 2: THE PROBABILITY OF THE SEQUENCE

The Forward Algorithm

Given x, compute $p(x) =$ sum over all state paths s of $p(x, s)$ - the total probability over every path, not just the best one.

Define $F_l(i) = p(x_1, \ldots, x_i\ ;\ S_i = l)$ – the probability of seeing $x_1, \ldots, x_i$ and the hidden state at position i is l.

Use the same recursive structure as Viterbi, but sum instead of max:

$$F_l(i) = \ e_l(x_i) \sum_k F_k(i-1).m_{kl}$$

Initialise $F_0(0) = 1, F_k(0) = 0$ for $k! = 0$, set m_{0l} appropriately

Result: $p(x)\ = \sum_k F_k(L)$

Worked Example: Decoding the Dishonest Casino

States F (fair coin) and B (biased)

Transitions $m_{FF} = 0.75, m_{FB} = 0.25, m_{BF} = 0.25, m_{BB} = 0.75$

Emissions $e_F(H) = 0.5, e_F(T) = 0.5, e_B(H) = 0.75, e_B(T) = 0.25$

Equal start probabilities $(F_0(0) = 1, m_{0F} = m_{0B} = 0.5)$

Observed sequences $\boldsymbol{x} = TTHHT$

State	t1: T	t2: T	t3: H	t4: H	t5: T
F	$0.5 \cdot 0.5 = 0.25$	$0.5(0.25 \cdot 0.75 + 0.125 \cdot 0.25)$			
B	$0.5 \cdot 0.25 = 0.125$	$0.25(0.125 \cdot 025 + 0.125 \cdot 0.75)$			

Worked Example: Decoding the Dishonest Casino

States F (fair coin) and B (biased)

Transitions $m_{FF} = 0.75, m_{FB} = 0.25, m_{BF} = 0.25, m_{BB} = 0.75$

Emissions $e_F(H) = 0.5, e_F(T) = 0.5, e_B(H) = 0.75, e_B(T) = 0.25$

Equal start probabilities $(F_0(0) = 1, m_{0F} = m_{0B} = 0.5)$

Observed sequences $\boldsymbol{x} = TTHHT$

State	t1: T	t2: T	t3: H	t4: H	t5: T
F	**0.25**	0.5(0.25·0.75 + 0.125·0.25) = **0.109375**	0.5(0.109375·0.75 + 0.0390625·0.25) = **0.045898**	0.5(0.045898·0.75 + 0.042480·0.25) = **0.022522**	0.5(0.022522·0.75 + 0.032501·0.25) = **0.012508**
B	**0.125**	0.25(0.25·0.25 + 0.125·0.75) = **0.0390625**	0.75(0.109375·0.25 + 0.0390625·0.75) = **0.042480**	0.75(0.045898·0.25 + 0.042480·0.75) = **0.032501**	0.25(0.022522·0.25 + 0.032501·0.75) = **0.007502**

Summing the last column gives p(x)=0.02001

Worked Example: Decoding the Dishonest Casino

States F (fair coin) and B (biased)

Transitions $m_{FF} = 0.75, m_{FB} = 0.25, m_{BF} = 0.25, m_{BB} = 0.75$

Emissions $e_F(H) = 0.5, e_F(T) = 0.5, e_B(H) = 0.75, e_B(T) = 0.25$

Equal start probabilities $(F_0(0) = 1, m_{0F} = m_{0B} = 0.5)$

Observed sequences $\boldsymbol{x} = TTHH\boldsymbol{H}$

State	t1: T	t2: T	t3: H	t4: H	t5: H
F	**0.25**	0.5(0.25·0.75 + 0.125·0.25) = **0.109375**	0.5(0.109375·0.75 + 0.0390625·0.25) = **0.045898**	0.5(0.045898·0.75 + 0.042480·0.25) = **0.022522**	0.5(0.022522·0.75 + 0.032501·0.25) = **0.012508**
B	**0.125**	0.25(0.25·0.25 + 0.125·0.75) = **0.0390625**	0.75(0.109375·0.25 + 0.0390625·0.75) = **0.042480**	0.75(0.045898·0.25 + 0.042480·0.75) = **0.032501**	0.75(0.022522·0.25 + 0.032501·0.75) = **0.022505**

Summing the last column gives p(x)=0.03501

Worked Example: Decoding the Dishonest Casino

States F (fair coin) and B (biased)

Transitions $m_{FF} = 0.75, m_{FB} = 0.25, m_{BF} = 0.25, m_{BB} = 0.75$

Emissions $e_F(H) = 0.5, e_F(T) = 0.5, e_B(H) = 0.75, e_B(T) = 0.25$

Equal start probabilities $(F_0(0) = 1, m_{0F} = m_{0B} = 0.5)$

Observed sequences

- $\boldsymbol{x} = TTHH\boldsymbol{T}, p(x) = 0.02001$
- $\boldsymbol{x} = TTHH\boldsymbol{H}, p(x) = 0.03501$

If a fair dice alone was used, then the probability of either sequence is $2^{-5} = 0.03125$

The backward algorithm, combined with the forward algorithm

PROBLEM 3: POSTERIOR STATE PROBABILITIES

The Backward Algorithm

We want, for each i, k: $p(S_i = k \mid x)$.

Define $B_l(i) = p(x_{i+1}, \ldots, x_L \mid S_i = l)$ – the probability of seeing $x_{i+1}, \ldots, x_L$ and the hidden state at site i is l.

Recursion, working backwards from the end of the sequence:

$$B_l(i) = \sum_k m_{lk} \, . \, e_k(x_{i+1}) . \, B_k(i+1)$$

Initialise $B_l(L) = 1$ for every state l, then compute $i = L-1, L-2, \ldots, 1$

Putting Forward and Backward Together

Useful factorisation: $p(x, S_i = l) = F_l(i).B_l(i)$

1. Run the forward algorithm once, storing $F_l(i)$ for every i, l
2. Run the backward algorithm once, storing $B_l(i)$ for every i, l
3. For any i, l:

$$p(S_i = l \mid x) = \frac{p(x, S_i = l)}{p(x)} = \frac{F_l(i)\,.\,B_l(i)}{p(x)}$$

Each algorithm does $O(m^2)$ work per position: $O(m^2\ L)$ time, $O(mL)$ space - same complexity class as Viterbi

Worked Example: Decoding the Dishonest Casino

States F (fair coin) and B (biased)

Observed sequences $\boldsymbol{x} = TTHHT$

Backward algorithm giving $B_l(i)$

State	t1: T	t2: T	t3: H	t4: H	t5: T
F	0.75·0.5·0.126709 + 0.25·0.25·0.157471 = **0.057358**	0.75·0.5·0.222656 + 0.25·0.75·0.230469 = **0.126709**	0.75·0.5·0.4375 + 0.25·0.75·0.3125 = **0.222656**	0.75·0.5·1 + 0.25·0.25·1 = **0.4375**	**1**
B	0.25·0.5·0.126709 + 0.75·0.25·0.157471 = **0.045364**	0.25·0.5·0.222656 + 0.75·0.75·0.230469 = **0.157471**	0.25·0.5·0.4375 + 0.75·0.75·0.3125 = **0.230469**	0.25·0.5·1 + 0.75·0.25·1 = **0.3125**	**1**

Check: p(x) = 0.5·0.5·0.057358 + 0.5·0.25·0.045364 = 0.02001, which matches the forward result.

Worked Example: Decoding the Dishonest Casino

States F (fair coin) and B (biased)

Observed sequences $\boldsymbol{x} = TTHHT$

Combining to give $p(S_i = l \mid x)$

State	t1: T	t2: T	t3: H	t4: H	t5: T
F	0.25·0.057358/P = **0.7166**	0.109375·0.126709/P = **0.6926**	0.045898·0.222656/P = **0.5107**	0.022522·0.4375/P = **0.4924**	0.012508·1/P = **0.6251**
B	0.125·0.045364/P = **0.2834**	0.0390625·0.157471/P = **0.3074**	0.042480·0.230469/P = **0.4893**	0.032501·0.3125/P = **0.5076**	0.007502·1/P = **0.3749**

Worked Example: Decoding the Dishonest Casino

States F (fair coin) and B (biased)

Observed sequences $\boldsymbol{x} = TTHHT$

State	t1: T	t2: T	t3: H	t4: H	t5: T
F	0.7166	0.6926	0.5107	0.4924	0.6251
B	0.2834	0.3074	0.4893	0.5076	0.3749

Observed sequences $\boldsymbol{x} = TTHHH$

State	t1: T	t2: T	t3: H	t4: H	t5: T
F	0.7044	0.6658	0.4455	0.3618	0.3572
B	0.2956	0.3342	0.5545	0.6382	0.6428

Applications of Decoding and Evaluation

- Gene finding: decode the most probable arrangement of exons, introns and intergenic states along a genome
- CpG-island detection: decode (or use posteriors) to locate island boundaries within one long sequence
- Profile HMMs: evaluate $p(x)$ to score how well a new protein sequence fits a known family
- All three algorithms are building blocks used throughout the rest of applied bioinformatics, not just in this module's small examples

Summary and Looking Ahead

Viterbi finds the single most probable hidden state path in $O(m^2\ L)$ time; forward computes $p(x)$; backward (with forward) gives posterior state probabilities

All three assumed the HMM's parameters (M, E) were already known

Next lecture: how to estimate m_{kl} and $e_k(b)$ from data - by counting when the state path is known, and by Expectation-Maximisation (Baum-Welch) when it is not